Cheatsheet: Interactive Code with Pyodide

Running Python live in the browser

1 — Install & enable

> ⚠ HTML only
>
> Interactive code needs **HTML output** (`format: html`). It works in websites and books, but not in PDF or Word — see the Exercise Sheet cheatsheet for code that has to work everywhere.

`pyodide-interaktiv` (this tutorial's fork, adding real `input()`, animations, a Stop button, and AI feedback over the base `coatless-quarto/pyodide` extension) is currently a **private** repo:

Repo access	Install
No access (for now)	Copy `_extensions/Erasmus-CTM/pyodide-interaktiv/` into your own project's `_extensions/` folder — no `quarto add` step needed
Once public	`quarto add Erasmus-CTM/pyodide-interaktiv` in the Terminal

Enable it in a page's YAML header:

```yaml
---
title: "My Page"
format: html
filters:
  - pyodide-interaktiv
---
```

2 — Writing interactive cells

Use a fence tagged `{pyodide-python}` instead of a regular code block:

```
x = [1, 2, 3, 4, 5]
print(sum(x))
```

Students can edit this code directly in the browser and run it with the **Run Code** button — nothing to install on their end.

Check it: run `quarto preview` — the page shows **PYODIDE STATUS: Ready!** once Python has loaded in the browser.

> Want graded exercises instead?
>
> `py-exercise` adds code blocks with hidden automated tests; `math-exercise` checks symbolic/numeric answers with SymPy. Both use the same install pattern as above — live examples are on the Example Page of this tutorial.

3 — What pyodide-interaktiv gives you

Because it runs Python in a **Web Worker** instead of on the browser's main thread:

Feature	What it means
Real `input()`	A cell can pause and wait for typed input without freezing the page
Animations	A Matplotlib `FuncAnimation` plays as an interactive player
Safe infinite loops	An accidental `while True:` no longer locks the tab — a Stop button cancels it
AI feedback	Every cell gets a Feedback button explaining what went wrong
Responsive page	Long calculations don't freeze the tab — they run off the main thread
Shared state	Cells on one page share a single Python session — reuse a variable or `import` from an earlier cell

Try `input()` and shared state live on the Code page, and the animation on the Visualizations page of the Example Page.

Full details: Interactive Python with Pyodide